

Module 25: Move Semantics

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Module 24 made copies correct; this module makes them unnecessary when the source is about to disappear anyway. Students learn to tell lvalues (things with names) from rvalues (temporaries), meet rvalue references, and write a move constructor and move assignment that steal a resource and leave the source empty but valid. The often-misunderstood `std::move` is presented honestly as a cast that grants permission to move. A measured demonstration shows `std::vector` copying instead of moving when a move constructor lacks `noexcept`. The module closes with the Rule of Five, move-only types, and the Rule of Zero again - which now gives fast moves for free.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Distinguish lvalues from rvalues, and explain why it is always safe to move from an rvalue.
- Write a move constructor and move assignment operator that leave the source empty but valid.
- Explain what `std::move` does and does not do, and what may be done with a moved-from object.
- Mark move operations `noexcept` and explain why `std::vector` depends on it.
- Apply the Rule of Five, design a move-only type, and prefer the Rule of Zero.

Prerequisites

- Module 24 (copy constructor, copy assignment, Rule of Three).
- Module 23 (temporaries and their lifetime).

Materials & Instructor Prep

- 25-move-semantics/examples/02_move_constructor.cpp - it prints array addresses; ask students to watch which addresses repeat.
- examples/04_noexcept_vector.cpp - the copy/move counts are the most persuasive evidence in the module.

Key Vocabulary

Term	Definition
lvalue	An expression with a name or identity that can be referred to again, such as a variable.
rvalue	A temporary value with no name that is about to disappear, such as a function's return value or the result of <code>a + b</code> .

Term	Definition
Rvalue reference (T&&)	A reference that binds only to rvalues, used to detect objects that are safe to steal from.
Move constructor / move assignment	Special member functions that transfer (steal) another object's resources instead of copying them, leaving the source empty but valid.
std::move	A cast that turns an expression into an rvalue, giving permission to move from it. It does not move anything itself.
noexcept	A promise that a function never throws. std::vector only moves elements during growth if their move constructor is noexcept.
Rule of Five	Destructor, copy constructor, copy assignment, move constructor and move assignment: if a class defines one, think about all five.

Lesson Outline & Instructor Talking Points

1. Why move? The photocopy analogy (10 min)

- Use the box-of-exam-scripts analogy from the README: copying photocopies every script; moving hands over the box. Moving is only fair if the first teacher was going home anyway.
- Draw the COPY vs MOVE pointer diagram next to Module 24's shallow-copy diagram and ask what makes a move safe when a shallow copy wasn't (the source gives up ownership).

2. lvalues, rvalues and && (15 min)

- Run examples/01_lvalues_rvalues.cpp after students predict each line. The last line - an rvalue reference variable being an lvalue - surprises nearly everyone.

Instructor note: Keep the definitions practical: 'has a name, can be used again' vs 'a temporary nobody can refer to again'. The formal value categories are not needed at this level.

3. Writing the move operations (25 min)

- Live-code the move constructor from examples/02_move_constructor.cpp. Remove the 'other.data = nullptr' line and ask what happens at destruction (double delete again).
- Run the example and trace the addresses: COPY makes a new address, MOVE reuses the old one, and the moved-from object reports size 0.
- Write move assignment: self-check, free our own data, steal, empty the source, return *this.

4. std::move and the moved-from state (10 min)

- Run examples/03_std_move.cpp. Stress the moved-from rules: you may destroy it or assign to it; you must not rely on its contents.
- Show the last section: binding std::move's result to a reference moves nothing - std::move is only a cast.

5. noexcept, the Rule of Five, move-only types (20 min)

- Run examples/04_noexcept_vector.cpp and read the counts aloud. Ask the class to explain the difference before you do.
- Run examples/06_move_only.cpp as the model for Module 26's unique_ptr, then examples/05_rule_of_zero.cpp to close on 'prefer the Rule of Zero'.

Instructor note: The noexcept demonstration is the moment the abstract rule becomes concrete. Let the numbers speak, then state the rule: always mark move operations noexcept.

Common Student Misconceptions

Misconception	How to Address It
"std::move moves the object."	std::move is only a cast to rvalue. The move constructor or move assignment does the actual moving - and if there isn't one, a copy happens instead.
"A moved-from object is destroyed or invalid."	It is still a valid object that will be destroyed normally. Its contents are unspecified, so you may only destroy it or give it a new value.
"return std::move(local); makes returning faster."	Returning a local already moves (or skips the copy entirely). Adding std::move can prevent that optimisation.
"If I write a destructor, I still get the move operations for free."	Declaring a destructor or copy operation stops the compiler generating move operations; moves quietly become copies.

Exercises & Module Project

Exercise: 25-move-semantics/exercises/exercise1.cpp - add noexcept move operations to Module 24's ReadingBuffer, record copy/move counts before and after for three tests, then remove noexcept from the move constructor and explain the change in the vector-growth test. Project (three tracks): Generic - building a 60-sheet term archive; EE - a by-value signal-processing pipeline; Biomedical - handing ECG recordings over to a central archive. Each adds the Rule of Five, measures copies vs moves, shows the moved-from state, and makes the Module 23 RAIL class move-only.

Facilitation guidance: Check that every move operation sets the source's pointer to nullptr, that both are noexcept, and that the RESULTS table at the top of main.cpp shows real before/after numbers. Run with -fsanitize=address to catch a missing nullptr. The stretch timing task is a good extension for faster students.

Discussion & Comprehension Check Questions

1. Why must a move constructor set the source's pointer to nullptr?

Guidance: Otherwise the source's destructor would delete the array that now belongs to the new object - a double delete. delete[] nullptr is defined to do nothing.

2. Why does std::vector copy its elements during growth if their move constructor isn't noexcept?

Guidance: If a move threw half-way through reallocation, the vector couldn't restore its original state. Copying leaves the original untouched, so it is the safe fallback.

3. When is it appropriate to make a type move-only?

Guidance: When it represents a unique resource that can be handed over but not duplicated - an open connection, a running device session, or sole ownership of a heap object.

Assessment Ideas

- Label each line of a short program as copy or move, including temporaries, `std::move` on a named object, and `std::move` on a const object.
- Ask students to explain in writing why their project's copy count dropped after adding move operations.

Answer Key Notes

See 25-move-semantics/exercises/solutions/exercise1_solution.cpp, which includes the expected counts as a RESULTS comment.

Wrap-Up & Bridge to Next Module

Bridge forward: "A move-only type that owns a heap object and deletes it automatically - that's exactly what `std::unique_ptr` is. Next module: smart pointers, and the end of writing delete yourself."

Instructor Tips

- If time is short, protect the move constructor (step 3) and the noexcept demonstration (step 5).
- Keep Module 24's diagrams visible - moving is easiest to understand as 'a copy that steals and then lets go'.